CLAUDESKILLS.CO.UK — BEGINNER SERIES

Prompting Basics

A beginner's guide to writing prompts that get you exactly what you need from Claude.

LEVEL
Beginner

CONTENTS

What's inside

01 Introduction — Why this matters 4
The one habit that changes everything

02 The Anatomy of a Good Prompt 5
Context · Task · Format · Constraints

03 The Formula 6
Putting the four parts together

04 Vague vs. Clear — Side by Side 7
Why specific prompts produce specific answers

05 Six Prompt Patterns to Copy 8
Explain · Write · Fix · Review · Refactor · Brainstorm

06 How to Iterate 10
When the first answer isn't right

07 Common Beginner Mistakes 11
What to avoid and what to do instead

08 Key Takeaways & Summary 12
Everything distilled

09 Quick Reference — Tear-Out Sheet 13
All six patterns on one page

THE GUIDE

Prompting Basics

You get what you ask for. This guide teaches you how to ask in a way that gets you exactly what you need — no special knowledge required.

Why this matters

Most beginners think they're bad at using Claude when really they're just asking in a way Claude doesn't understand. The good news: this is fixable. Once you understand the pattern, every answer gets better.

> ***You get what you ask for. If you ask Claude vaguely, you'll get a vague answer. If you ask clearly, you'll get a clear answer.***

This guide teaches you how to ask Claude questions in a way that gets you exactly what you need. No special knowledge required — just clear thinking and a few patterns to copy.

THE ANATOMY OF A GOOD PROMPT

Every prompt has four parts. Not all prompts need all four — but knowing about them helps you build better ones.

1. **Context** — What should Claude know? Your role, the domain, your constraints, your goal. Without context, Claude might assume you're an expert or give an answer that doesn't match your situation.
2. **Task** — What do you want Claude to do? Use a verb: Explain, Write, Fix, Review, Brainstorm. A clear task tells Claude exactly what to produce.
3. **Format** — How should Claude give the answer? Length, structure, tone, medium. Saves you editing time — just ask for what you want upfront.
4. **Constraints** — What should Claude avoid or stick to? These guardrails prevent Claude from going down the wrong path before it starts.

> **GOOD TO KNOW**
>
> You don't need all four every time. A simple question doesn't need format or constraints. Add parts as the task gets more complex.

What each part looks like in practice

Good context

CONTEXT EXAMPLES

✓ "I'm a complete beginner to JavaScript. I don't know what promises or async/await are."
✓ "We're building a tool that processes Excel files. Files can be up to 100MB with 500,000 rows."
✓ "I'm writing a marketing email. Our audience is busy founders who hate hype."

Clear tasks vs vague tasks

✗ VAGUE

Tell me about React
Help me with this code
What's wrong here?
Say something about email

✓ CLEAR

Explain how React hooks work
Fix this error: [error message]
Review this code for security issues
Write a marketing email for founders

Format specifications

FORMAT EXAMPLES

"Give me a 3-step guide as a numbered list"
"Write it in under 150 words"
"Explain it like I'm 12 years old"
"Give me Python code that runs without dependencies"

Constraints

CONSTRAINT EXAMPLES

"Only use Python built-in libraries — no pip packages"
"Keep it under 10 lines of code"
"Make it work on Mac and Windows"
"Don't assume the reader knows SQL"

The formula

Put the four parts together in one message:

[Context]. [Task]. [Format]. [Constraints].

Three worked examples

EXAMPLE 1 — LEARNING A CONCEPT

I'm a beginner learning JavaScript and I've never used functions before. **Explain what a JavaScript function is.** Keep it to one paragraph. Use simple words and give one example.

EXAMPLE 2 — WRITING CODE

I'm building a CLI tool in Python for beginners. I need to validate email addresses. **Write a function that checks if an email is valid.** Use only Python built-in libraries. Keep comments simple. Test it with "test@example.com".

EXAMPLE 3 — NON-TECHNICAL QUESTION

I'm a non-technical founder who just learned what an API is. **What's the difference between an API key and a password?** Explain in under 100 words using everyday language. Use an analogy if it helps.

◆ **TIP**

The highlighted part of each example is the **task** — everything else is context, format, or constraints. If you can only add one thing to a vague prompt, make it a clearer task verb.

Vague vs. clear — side by side

Situation	Vague prompt	Clear prompt
Learning	"Tell me about React"	"Explain React hooks to a JS beginner in one paragraph with one example"
Debugging	"Fix this"	"Fix this error: [paste]. Here's my code. I've already tried X."
Writing code	"Help with email"	"Write a Python email validator using only built-in libraries"
Feedback	"Is this good?"	"Review this for security issues — I'm a beginner, explain in plain English"
Ideas	"Give me blog ideas"	"Brainstorm 5 blog titles about productivity. Skip 'tips and tricks' formats"

Clear prompts are not longer — they're more **specific**. They answer the questions Claude would otherwise have to guess: Who are you? What exactly do you want? What constraints matter?

THE COMPLETE PROMPT CHECKLIST

Before you send a prompt, run through this:

- [ ] **Context?** Does Claude know who I am and what my situation is?
- [ ] **Task?** Is there a clear verb — Explain, Write, Fix, Review, Brainstorm?
- [ ] **Format?** Did I say how long and what structure I want?
- [ ] **Constraints?** Did I mention any important "don'ts"?
- [ ] **Specific?** Could I replace any vague words with specific ones?

Six prompt patterns to copy

These six patterns cover 90% of situations. Copy the template and fill in the gaps.

1 Explain

For learning

```
I don't understand [concept]. Explain it like I have no background in [domain]. Use an analogy or real-world example if it helps.
```

Example: *"I don't understand async/await in JavaScript. Explain it like I've only written synchronous code. Use a real-world analogy."*

2 Write

For generating code or content

```
Write [what you want] that [requirements]. It should [additional constraints]. Test it with [example input].
```

Example: *"Write a Python function that takes a list of numbers and returns the average. Handle empty lists. Test with [1, 2, 3] and []."*

3 Fix

For debugging

```
This [code/text] isn't working. The problem is [what goes wrong]. I've already tried [what you've tried]. Fix it and explain what the problem was.
```

Example: *"This JS function isn't working — when I pass a date as a string it says 'undefined'. I've tried new Date(). Fix it and explain what was happening."*

Six prompt patterns (continued)

4 Review

For getting feedback

```
Review this [code/content] for [specific thing]. I'm [your skill level]. Focus on [what matters to you]. Be honest but kind.
```

Example: *"Review this JavaScript code for performance issues. I'm a beginner — explain fixes in simple language. Focus on anything that would slow down a real user."*

5 Refactor

For improving existing code

```
Refactor this [code] to be [simpler/faster/more readable]. Keep it [constraints]. Show me the before and after.
```

Example: *"Refactor this Python code to be more readable. Keep it working on Python 3.8+. Show me before/after with a brief explanation of what changed."*

6 Brainstorm

For generating ideas

```
Brainstorm [topic]. Give me [number] ideas. I'm looking for [what matters]. Skip [what you don't want].
```

Example: *"Brainstorm 5 blog post ideas about productivity. I'm looking for fresh, counterintuitive angles. Skip 'tips and tricks' formats."*

◆ HOW TO USE THESE

Copy the template word for word. Fill in the brackets. Send it. After a few tries you'll stop thinking about the structure — it becomes natural. Even experienced Claude users return to these when stuck.

How to iterate

You won't always get it perfect on the first try. That's normal. Here's how to fix it without starting over.

When the answer is too long

✗ **BAD**

That's too long.

✓ **GOOD**

That's helpful but too long. Can you give me just the key 3 points as bullet points?

When the answer is too simple

✗ **BAD**

I need more detail.

✓ **GOOD**

That helps, but I still don't understand step 2. Walk me through how the loop works, line by line.

When the answer uses jargon you don't know

✗ **BAD**

Use simpler words.

✓ **GOOD**

That helps, but I don't know what "promise" means. Explain that one word like I'm seeing it for the first time.

When the code doesn't work

✗ **BAD**

This code doesn't work.

✓ **GOOD**

I ran this and got this error: [paste full error]. What's wrong?

When to edit vs. start fresh

EDIT YOUR MESSAGE WHEN:

You spotted a typo, missing word, or awkward phrasing. Use the pencil icon — Claude responds to what you actually meant.

START FRESH WHEN:

You're going in a completely different direction. Use /clear in Claude Code or start a new chat in claude.ai.

Common beginner mistakes

1 Burying the main question

✗

```
I've been learning Python for a week… I
read a blog post yesterday… Can you tell me
about loops?
```

✓

```
Explain Python loops. I've been learning
for a week and have only done simple
scripts.
```

Main question first, then context.

2 Asking multiple things at once

✗

```
How do I set up a project? Should I use
Git? What folder structure? Is npm better
than yarn?
```

✓

```
I'm starting a JS project. What's the best
folder structure for a beginner? (I'll ask
about Git next.)
```

One clear question at a time.

3 Not giving Claude anything to work with

✗

```
How do I fix this?
[no code shown]
```

✓

```
How do I fix this? Here's the code: [code].
Here's what happens when I run it: [error].
```

Claude can only debug what it can see.

4 Being too polite and vague

✗

```
Hi Claude, I hope you're having a good day.
I was wondering if you could maybe help me
think about how I might possibly use an
API?
```

✓

```
I need to use an API but I'm not sure where
to start. I know JavaScript but have never
used one. Explain and show me a simple
example.
```

Direct is not rude. Clear is kind.

★ KEY TAKEAWAYS

1. **Context first, task clear.** Tell Claude who you are and what you need. A clear verb (Explain, Write, Fix) sets the direction immediately.
2. **Specify the format you want.** Bullet points, numbered list, code block, under 150 words — say it upfront. Stop reformatting after the fact.
3. **Edit, don't just follow up.** If your prompt was unclear, use the edit button. Cleaner history, better answers, no wasted context.
4. **One question at a time.** Multiple questions in one message almost always produces weaker answers to each. Split them up.
5. **Use the six patterns.** Explain, Write, Fix, Review, Refactor, Brainstorm. Copy, fill in the blanks, send. It becomes natural fast.

GUIDE SUMMARY

Prompting Basics

§1 **Why this matters**
Bad results usually come from unclear prompts — not Claude.

§2 **The four parts**
Context · Task · Format · Constraints — the anatomy of any good prompt.

§3 **The formula**
[Context]. [Task]. [Format]. [Constraints]. — with three worked examples.

§4 **Six patterns to copy**
Explain, Write, Fix, Review, Refactor, Brainstorm — templates for every situation.

§5 **Iterating & common mistakes**
How to follow up well, when to edit vs. restart, and what beginners get wrong.

→ **Next in the series: Managing Context & Usage Limits**

Quick reference

Tear this page out. Use it every time.

THE FORMULA

```
[Context]. [Task].
[Format]. [Constraints].
```

THE CHECKLIST

- ☐ Context given?
- ☐ Clear task verb?
- ☐ Format specified?
- ☐ Constraints mentioned?
- ☐ Specific enough?

SIX PATTERNS — COPY & FILL IN THE BLANKS

EXPLAIN

```
I don't understand [concept]. Explain it like I have no background in [domain]. Use an analogy if it helps.
```

WRITE

```
Write [what you want] that [requirements]. It should [constraints]. Test it with [example].
```

FIX

```
This [code] isn't working. The problem is [what goes wrong]. I've tried [X]. Fix it and explain what was wrong.
```

REVIEW

```
Review this [code/content] for [specific thing]. I'm [skill level]. Focus on [what matters]. Be honest.
```

REFACTOR

```
Refactor this [code] to be [simpler/faster/more readable]. Keep it [constraints]. Show before and after.
```

BRAINSTORM

```
Brainstorm [topic]. Give me [number] ideas. I'm looking for [what matters]. Skip [what you don't want].
```

You're ready. Go ask Claude something.

Good prompts aren't magic. They're just clear thinking written down. Start with the formula. Use the patterns. After a few tries, it becomes second nature.

claudeskills.co.uk

More guides in the series →